MNIST Neural Network: How It Works

Contents

Notation (list of symbols)

P	one image, raw intensities; 28×28, entries $\{0, \dots, 255\}$.
$\tilde{P}$	normalized image; 28×28, entries $[0, 1]$.
x	one flattened, normalized image; $\mathbb{R}^{784}$ (column).
X	batch of inputs, rows = examples; $\mathbb{R}^{N \times 784}$.
N	mini-batch size; scalar.
t	true digit label; scalar $\{0, \dots, 9\}$.
y	one-hot target for one image; $\mathbb{R}^{10}$.
$\hat{y}$	predicted class distribution (reserved); $\mathbb{R}^{10}$.
Y	batch of one-hot targets, rows = examples; $\mathbb{R}^{N \times 10}$.
H	hidden-layer width; scalar ($= 128$).
$W^{(1)}$	layer-1 weights, input $\to$ hidden; $\mathbb{R}^{784 \times 128}$.
$b^{(1)}$	layer-1 biases, one per hidden unit; $\mathbb{R}^{128}$.

$z^{(1)}$	hidden pre-activation, one example; $\mathbb{R}^{128}$.
$Z^{(1)}$	hidden pre-activation, whole batch; $\mathbb{R}^{N\times 128}$.
$a^{(1)}$	hidden activation, post-ReLU, one example; $\mathbb{R}^{128}$.
$A^{(1)}$	hidden activation, post-ReLU, whole batch; $\mathbb{R}^{N\times 128}$.
$W^{(2)}$	layer-2 weights, hidden $\to$ output; $\mathbb{R}^{128\times 10}$.
$b^{(2)}$	layer-2 biases, one per class; $\mathbb{R}^{10}$.
$z^{(2)}$	output logits, one example; $\mathbb{R}^{10}$.
$Z^{(2)}$	output logits, whole batch; $\mathbb{R}^{N\times 10}$.
$\hat{Y}$	predicted class distributions, whole batch; $\mathbb{R}^{N\times 10}$.
$\mathcal{L}$	mean cross-entropy loss over the batch, the training objective; scalar.
$g^{(2)}$	gradient of loss w.r.t. the layer-2 pre-activation (logits), $= \frac{1}{N}(\hat{Y} - Y)$; $\mathbb{R}^{N\times 10}$.
$g^{(1)}$	gradient of loss w.r.t. the layer-1 pre-activation; $\mathbb{R}^{N\times 128}$.
η	learning rate (gradient-descent step size); positive scalar, running value 0.1.
E	number of training epochs (full passes over the data); positive integer.

1 Data representation

MNIST is 70,000 grayscale images of handwritten digits, split 60,000 training / 10,000 test. Each image is 28×28 pixels; each pixel is an 8-bit intensity in $\{0, \dots, 255\}$ (0 = black, 255 = white). The label is the digit shown, one of ten classes.

Everything downstream is linear algebra, so the first job is to turn an image and its label into vectors with fixed, known shapes.

1.1 One image

Write a single image as a matrix

$$P \in \{0, \dots, 255\}^{28\times 28}, \qquad P_{rc} = \text{intensity at row } r, \text{ column } c.$$

Here $r, c \in \{0, \dots, 27\}$ are 0-indexed to match array code (NumPy / PyTorch); nothing mathematical depends on the offset.

Normalize to $[0, 1]$ by dividing by the maximum intensity:

$$\tilde{P}_{rc} = \frac{P_{rc}}{255} \in [0, 1].$$

Reason: keeps inputs $O(1)$ so weight initialization and learning rates behave predictably. $\tilde{P}$ has the same shape, 28×28.

1.2 Flatten to a vector

A dense (fully-connected) layer treats its input as a flat vector, so unroll the matrix *row-major* into

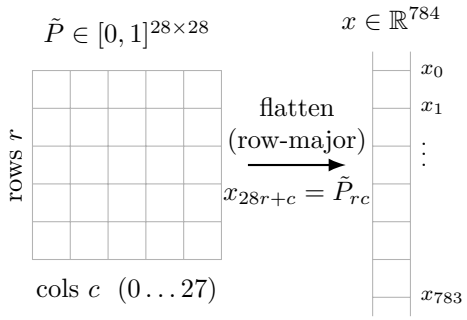
$$x \in \mathbb{R}^{784}, \qquad 784 = 28 \times 28,$$

with the index map

$$x_{\,28r+c} = \tilde{P}_{rc}, \qquad r, c \in \{0, \ldots, 27\}.$$

Sanity check: as (r, c) range over $0 \ldots 27$, the index $28r + c$ runs over $0 \ldots 783$ with no gaps or collisions — exactly the 784 components. Convention: x is a *column* vector, i.e. $x \in \mathbb{R}^{784\times 1}$.

Flattening discards the 2-D spatial layout (which pixels are neighbours). A plain feed-forward net does not use that structure anyway; convolutional nets do, and keep the matrix form. We are building the feed-forward version.



1.3 The label

The raw label is the target digit

$$t \in \{0, \ldots, 9\}.$$

For a classifier with one output per class, encode it *one-hot*:

$$y \in \mathbb{R}^{10}, \qquad y_i = \begin{cases} 1 & i = t \\ 0 & i \neq t \end{cases}, \qquad i \in \{0, \ldots, 9\}.$$

So digit 3 becomes $y = (0, 0, 0, 1, 0, 0, 0, 0, 0, 0)^\top$.

Notation flag. y here is the *true* target. The network's predicted distribution over classes will be a *different* 10-vector; we reserve $\hat{y} \in \mathbb{R}^{10}$ for that. Do not let y and $\hat{y}$ collide. Also: the superscript in $x^{(n)}$ below means "example n," not a power — it is always parenthesized to mark the difference.

1.4 A mini-batch

We train on mini-batches, not one image at a time. Stack N examples (the batch size) as rows:

$$X \in \mathbb{R}^{N\times 784}, \quad \text{row } n = \left(x^{(n)}\right)^\top; \qquad Y \in \mathbb{R}^{N\times 10}, \quad \text{row } n = \left(y^{(n)}\right)^\top.$$

N is a hyperparameter (e.g. 64). Examples-as-rows is the convention used throughout; it makes the forward pass XW rather than WX, which we will see next step.

1.5 Symbols

Persistent symbols introduced here — P, $\tilde{P}$, x, X, Y, N, t, y, $\hat{y}$ — live in the global list of symbols at the front; click any to jump. The index letters below are *local to this section*: later sections may reuse the same letters for other roles.

Local index	Meaning (this section)
r, c	pixel row / column, $0, \ldots, 27$
i	class index, $0, \ldots, 9$
n	example index in the batch, $0, \ldots, N-1$

Where this is heading

Running example for the whole document: one hidden layer of 128 ReLU units, softmax over 10 outputs, trained with mini-batch SGD. We state each piece when we reach it.

2 The first linear layer

The network we are building is

$$\underbrace{784}_{\text{input}} \longrightarrow \underbrace{128}_{\text{hidden, ReLU}} \longrightarrow \underbrace{10}_{\text{output, softmax}} .$$

This step is the first arrow: the affine map (linear part + bias) from the 784-dim input to a 128-dim hidden *pre-activation*. The ReLU nonlinearity is the next step; here we stop just before it.

Assumptions fixed now:

- hidden width $H = 128$ (a hyperparameter);
- examples are rows, as in step 1 ($X \in \mathbb{R}^{N \times 784}$).

2.1 What one hidden unit computes

A hidden unit forms a weighted sum of all 784 inputs, plus a bias. For hidden unit k and a single image $x \in \mathbb{R}^{784}$ (step 1):

$$z_k^{(1)} = \sum_{j=0}^{783} x_j \, W_{jk}^{(1)} + b_k^{(1)}, \qquad k \in \{0, \ldots, 127\}.$$

New symbols:

- $W_{jk}^{(1)}$ — weight connecting input feature j to hidden unit k; $j \in \{0, \ldots, 783\}$, $k \in \{0, \ldots, 127\}$.
- $b_k^{(1)}$ — bias of hidden unit k (one scalar per unit).
- $z_k^{(1)}$ — pre-activation of hidden unit k (the sum, before ReLU).

That is all a unit does: a dot product with its own weight column, shifted by a bias.

Notation flag (superscripts). The superscript (1) labels the *layer* (layer 1) — a literal integer, not a power. It is a *different* thing from the example index (n) in $x^{(n)}$, which ranges over the batch. Convention for this document: parenthesized *numbers* $(1), (2)$ = layer; a parenthesized *letter* (n) = example.

2.2 Stacking the units: one example

Collect the 128 units' weights into a matrix and the biases into a vector:

$$W^{(1)} \in \mathbb{R}^{784\times 128}, \qquad b^{(1)} \in \mathbb{R}^{128}.$$

Column k of $W^{(1)}$ is hidden unit k's weight vector. Writing the single image as a *row* $x^\top \in \mathbb{R}^{1\times 784}$, all 128 sums at once are

$$z^{(1)\top} = x^\top W^{(1)} + b^{(1)\top} \in \mathbb{R}^{1\times 128},$$

so $z^{(1)} \in \mathbb{R}^{128}$ is the hidden pre-activation vector for this example.

Why $x^\top W^{(1)}$ and not $(W^{(1)})^\top x$? Same computation, different bookkeeping: with the example as a *row* you right-multiply by $W^{(1)}$; with it as a *column* you would write $z^{(1)} = (W^{(1)})^\top x + b^{(1)}$. We keep examples as rows (step 1), so the row form is what we use.

2.3 Batching

Stack all N examples as the rows of X and apply the same map to every row at once:

$$Z^{(1)} = X\,W^{(1)} + b^{(1)} \in \mathbb{R}^{N\times 128}.$$

Element-wise,

$$Z^{(1)}_{nk} = \sum_{j=0}^{783} X_{nj}\,W^{(1)}_{jk} + b^{(1)}_k, \qquad n \in \{0,\dots,N-1\},\ k \in \{0,\dots,127\}.$$

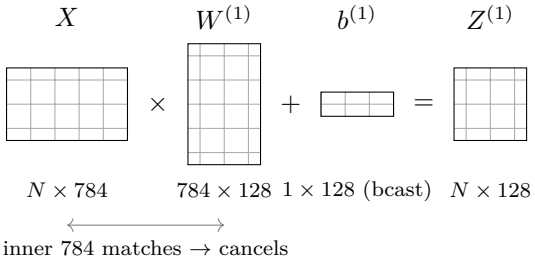
Row n of $Z^{(1)}$ is the hidden pre-activation for example n.

Bias broadcasting. $XW^{(1)}$ has shape $N\times 128$ but $b^{(1)}$ is a single 128-vector. The "$+\,b^{(1)}$" means add $b^{(1)}$ to *every row* — the same bias for all examples. (NumPy/PyTorch do this by broadcasting; mathematically $Z^{(1)}_{nk} = (XW^{(1)})_{nk} + b^{(1)}_k$, with $b^{(1)}_k$ independent of n.)

Shape sanity-check.

$$\underset{N\times 784}{X} \cdot \underset{784\times 128}{W^{(1)}} = \underset{N\times 128}{XW^{(1)}},$$

the inner 784 must match and cancels; the bias is 128 wide, matching the columns; the result is $N \times 128$ — one row per example, one column per hidden unit.



Parameter count. This layer has $784 \times 128 = 100\,352$ weights plus 128 biases $= 100\,480$ learnable parameters. (The later $128 \to 10$ layer will add only $128 \times 10 + 10 = 1\,290$.)

Notation flag (case). Lowercase $z^{(1)}$ = one example's pre-activation vector ($\in \mathbb{R}^{128}$); uppercase $Z^{(1)}$ = the whole batch ($\in \mathbb{R}^{N \times 128}$) — same lower/upper convention as x/X and y/Y.

2.4 Initialization (brief)

$W^{(1)}$ and $b^{(1)}$ are exactly what training adjusts. They start at: $b^{(1)} = 0$, and $W^{(1)}$ small random values (the precise scheme — He initialization, suited to ReLU — we set in the training step). The point for now: they are parameters, with the shapes above.

2.5 Symbols

New persistent symbols this step — H, $W^{(1)}$, $b^{(1)}$, $z^{(1)}$, $Z^{(1)}$ — are added to the global list of symbols. All persistent symbols from step 1 still stand. The index letters below are *local to this section.*

Local index	Meaning (this section)
j	input-feature index, $0, \ldots, 783$
k	hidden-unit index, $0, \ldots, 127$
n	example index in the batch, $0, \ldots, N-1$

Next

Apply ReLU element-wise to $Z^{(1)}$ (step 3) to get the hidden activations $A^{(1)} = \max(0,\, Z^{(1)})$ — that nonlinearity is what lets the two linear layers represent more than a single linear map.

3 The ReLU activation

step 2 produced the hidden pre-activation $Z^{(1)}$. If we fed that straight into the next linear layer, depth would buy us nothing — so before the next layer we apply a nonlinearity, element by element.

3.1 Why a nonlinearity is needed

Suppose we skipped it and stacked two linear layers directly. With one example as a row $x^\top$ (step 2) and a second linear layer $W^{(2)} \in \mathbb{R}^{128 \times 10}$, $b^{(2)} \in \mathbb{R}^{10}$ (formalized next step):

$$\underbrace{\left(x^\top W^{(1)} + b^{(1)}\right)}_{\text{layer 1}} W^{(2)} + b^{(2)} = x^\top \underbrace{W^{(1)} W^{(2)}}_{W'} + \underbrace{b^{(1)} W^{(2)} + b^{(2)}}_{b'} = x^\top W' + b'.$$

Rules used: distributivity of matrix multiplication over addition, then associativity ($(x^\top W^{(1)})W^{(2)} = x^\top (W^{(1)} W^{(2)})$). The composition is again a single affine map, with $W' \in \mathbb{R}^{784 \times 10}$ and $b' \in \mathbb{R}^{10}$. So two linear layers collapse to one: no extra expressive power. A nonlinearity between them is what breaks the collapse.

3.2 Definition

ReLU (rectified linear unit) clamps negatives to zero, element by element. For one example,

$$a^{(1)} = \max\big(0,\, z^{(1)}\big) \in \mathbb{R}^{128}, \qquad a_k^{(1)} = \max\big(0,\, z_k^{(1)}\big), \quad k \in \{0, \dots, 127\}.$$

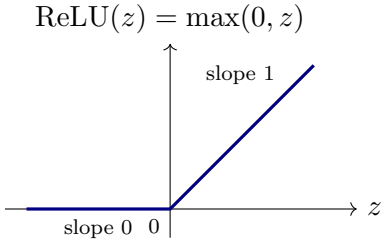
Applied to every entry of the batch pre-activation,

$$A^{(1)} = \max\big(0,\, Z^{(1)}\big) \in \mathbb{R}^{N\times 128}, \qquad A_{nk}^{(1)} = \max\big(0,\, Z_{nk}^{(1)}\big).$$

New symbols:

- $a^{(1)}$ — hidden activation (post-ReLU) for one example; $\mathbb{R}^{128}$.
- $A^{(1)}$ — the same for the whole batch; $\mathbb{R}^{N\times 128}$.

Shape check: ReLU is elementwise, so it cannot change shape — $A^{(1)}$ has exactly the shape of $Z^{(1)}$, namely $N \times 128$.



3.3 Derivative (needed for backprop)

ReLU is differentiable everywhere except at $z = 0$:

$$\frac{d}{dz}\max(0, z) = \begin{cases} 1 & z > 0, \\ 0 & z < 0, \end{cases} \qquad \text{undefined at } z = 0.$$

At the kink we pick a subgradient; the standard choice is 0 (using 1 works equally well in practice). **Flag:** this 0/1 value is the *entire* local gradient of the activation. In backprop it becomes an elementwise 0/1 *mask* with the shape of $Z^{(1)}$, gating which units pass gradient through; we formalize that mask in the backprop step.

3.4 Two practical notes

- Cheap: one comparison per entry — no exponentials, unlike sigmoid/tanh.
- Dead units: a unit whose pre-activation is negative for *every* input outputs 0 and (by the derivative above) receives zero gradient — it is "dead" and never recovers. Sensible initialization (the He scheme flagged in step 2) keeps this rare.

3.5 Symbols

New persistent symbols this step — $a^{(1)}$, $A^{(1)}$ — are added to the global list of symbols. The index letters below are *local to this section.*

Local index	Meaning (this section)
n	example index in the batch, $0, \ldots, N-1$
k	hidden-unit index, $0, \ldots, 127$

Next

Feed $A^{(1)}$ into the second linear layer ($128 \to 10$) to get the output logits, then turn those into a probability distribution $\hat{y}$ with softmax.

4 Sigmoid, and why ReLU instead

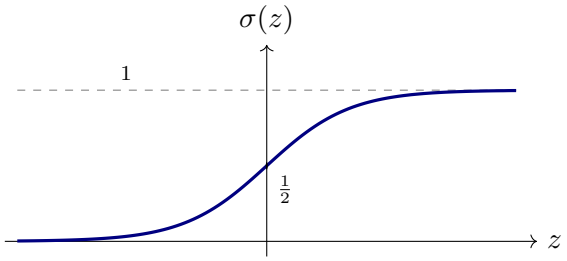
ReLU (step 3) is the modern default for hidden layers, but the *sigmoid* is the classic alternative. Understanding it explains *why* ReLU won, and sigmoid still shows up at output layers (binary classification, gating). Our network uses ReLU on $Z^{(1)}$ and softmax at the output, so sigmoid never enters the pipeline — this section is comparison only.

4.1 Definition

The logistic sigmoid squashes any real number into $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1).$$

Here σ is the function and $z \in \mathbb{R}$ a single scalar pre-activation (local to this section). It is smooth, monotonic increasing, S-shaped, with asymptotes $\sigma(z) \to 0$ as $z \to -\infty$ and $\sigma(z) \to 1$ as $z \to +\infty$, and $\sigma(0) = \frac{1}{2}$.



4.2 Derivative

A clean closed form, which is the reason sigmoid was historically convenient. Start from $\sigma(z) = (1 + e^{-z})^{-1}$ and differentiate:

$$\sigma'(z) = -(1 + e^{-z})^{-2} \cdot \frac{d}{dz}\big(1 + e^{-z}\big) \qquad \text{(chain rule, power rule)}.$$

The inner derivative is $\dfrac{d}{dz}(1 + e^{-z}) = -e^{-z}$ (chain rule on e^{-z}), so

$$\sigma'(z) = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2}.$$

Now factor to express it in σ itself:

$$\frac{e^{-z}}{(1+e^{-z})^2} = \underbrace{\frac{1}{1+e^{-z}}}_{\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1+e^{-z}}}_{1-\sigma(z)},$$

where the second factor is $1-\sigma(z)$ because $1-\sigma(z) = 1 - \frac{1}{1+e^{-z}} = \frac{e^{-z}}{1+e^{-z}}$ (common denominator). Hence

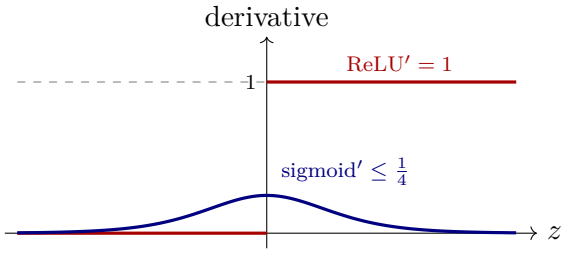
$$\boxed{\sigma'(z) = \sigma(z)\big(1-\sigma(z)\big)}$$

This is maximal at $z=0$: $\sigma(0) = \frac{1}{2}$ gives $\sigma'(0) = \frac{1}{4}$. So the slope is *never* larger than $\frac{1}{4}$, and it decays to 0 in both tails.

4.3 ReLU vs. sigmoid

	ReLU $\max(0,z)$	Sigmoid $\sigma(z)$
Range	$[0,\infty)$	$(0,1)$
Derivative	0 or 1	$\sigma(1-\sigma) \in (0, \frac{1}{4}]$
Saturates?	no (for $z>0$)	yes — both tails, gradient $\to 0$
Cost	one comparison	needs e^{-z}
Zero-centred out?	no (≥ 0)	no (>0)
Typical use	hidden layers	binary output / gates
Failure mode	dead units (step 3)	vanishing gradients

The decisive difference is the derivative. Backprop multiplies the per-layer derivatives together (chain rule — we will see this in the backprop step). With sigmoid every factor is $\leq \frac{1}{4}$, so through L stacked sigmoid layers the gradient is scaled by at most $(\frac{1}{4})^L$ — it vanishes exponentially with depth, and early layers barely learn. ReLU's derivative is exactly 1 on active units, so it introduces no such shrinkage; gradients reach early layers intact. That is why ReLU replaced sigmoid for hidden layers. The plot below contrasts the two derivatives:



4.4 Symbols

This section adds no persistent symbols. Local to it:

Local symbol	Meaning (this section)
σ	logistic sigmoid, $\sigma(z) = \dfrac{1}{1+e^{-z}}$
z	a single scalar pre-activation, $\in \mathbb{R}$

Back to the pipeline

Our hidden layer uses ReLU. Next we build the output layer ($128 \to 10$) and turn its logits into a probability distribution $\hat{y}$ with softmax.

5 Output layer and softmax

step 3 left us with the hidden activations $A^{(1)} \in \mathbb{R}^{N\times 128}$. Two things remain to produce a prediction: map the 128 hidden units to 10 class scores, then turn those scores into a probability distribution over the digits.

5.1 Second linear layer: logits

Exactly the same affine construction as step 2, now $128 \to 10$. These are the $W^{(2)}, b^{(2)}$ we promised in step 4:

$$W^{(2)} \in \mathbb{R}^{128\times 10}, \qquad b^{(2)} \in \mathbb{R}^{10}.$$

For the batch,

$$Z^{(2)} = A^{(1)}\, W^{(2)} + b^{(2)} \in \mathbb{R}^{N\times 10}, \qquad Z^{(2)}_{ni} = \sum_{k=0}^{127} A^{(1)}_{nk}\, W^{(2)}_{ki} + b^{(2)}_i.$$

Shape check: $(N \times 128)(128 \times 10) = N \times 10$; the bias is 10 wide, broadcast over the rows. Per example, $z^{(2)} \in \mathbb{R}^{10}$.

These ten numbers are the *logits*: raw, unnormalized scores. **Flag (logits vs. probabilities).** A logit $z^{(2)}_i$ can be any real number, positive or negative, and the ten do not sum to anything in particular. They are *not* probabilities yet — softmax fixes that.

5.2 Softmax: logits to a distribution

We want $\hat{y}_i$ to be the model's probability that the image is digit i: every entry ≥ 0, all summing to 1. Softmax produces exactly that from the logits:

$$\hat{y}_i = \frac{e^{z^{(2)}_i}}{\sum_{l=0}^{9} e^{z^{(2)}_l}}, \qquad i \in \{0, \ldots, 9\}.$$

Flag (two class indices). i is the class we are computing the probability *for* — a free index. l ranges over *all* classes in the denominator — a bound, summation index. Both run $0 \ldots 9$ but play different roles; do not conflate them.

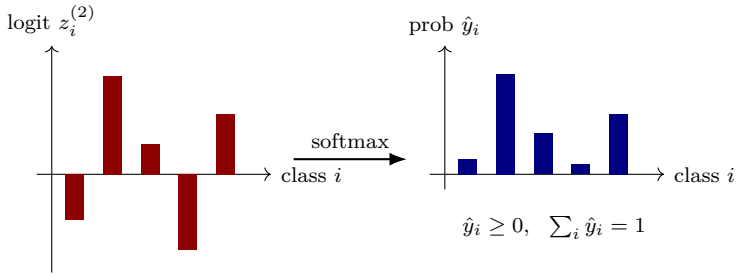
Why this is a valid distribution:

- Positive: $e^{(\cdot)} > 0$ always, so every $\hat{y}_i > 0$.
- Sums to one:

$$\sum_{i=0}^{9} \hat{y}_i = \sum_{i=0}^{9} \frac{e^{z^{(2)}_i}}{\sum_l e^{z^{(2)}_l}} = \frac{\sum_{i=0}^{9} e^{z^{(2)}_i}}{\sum_{l=0}^{9} e^{z^{(2)}_l}} = 1,$$

  pulling the common denominator out of the sum (it does not depend on i); numerator and denominator are then the same sum.

- Order-preserving: $e^{(\cdot)}$ is increasing, so the largest logit gives the largest probability — the predicted digit is $\arg\max_i z_i^{(2)} = \arg\max_i \hat{y}_i$.



5.3 Numerical stability (shift by the max)

Softmax is unchanged if we subtract any constant c from every logit:

$$\frac{e^{z_i - c}}{\sum_l e^{z_l - c}} = \frac{e^{z_i}\, e^{-c}}{e^{-c} \sum_l e^{z_l}} = \frac{e^{z_i}}{\sum_l e^{z_l}},$$

since e^{-c} factors out of the sum (constant-multiple rule) and cancels. Implementations take $c = \max_l z_l$, so the largest exponent becomes $e^0 = 1$ and e^z never overflows. Mathematically identical, numerically safe. (Here z abbreviates $z^{(2)}$.)

5.4 Batch form

Apply softmax to each row of $Z^{(2)}$ independently:

$$\hat{Y} = \text{softmax}\big(Z^{(2)}\big) \text{ (row-wise)} \in \mathbb{R}^{N \times 10},$$

where row n is example n's distribution $\hat{y}$ and sums to 1.

5.5 Symbols

New persistent symbols this step — $W^{(2)}$, $b^{(2)}$, $z^{(2)}$, $Z^{(2)}$, $\hat{Y}$ (and $\hat{y}$, reserved back in step 1 and now finally used) — are added to the global list of symbols. The index letters below are *local to this section.*

Local index	Meaning (this section)
n	example index in the batch, $0, \ldots, N-1$
k	hidden-unit index (summed in the matmul), $0, \ldots, 127$
i	class index, the one being computed, $0, \ldots, 9$
l	class index, summed in the softmax denominator, $0, \ldots, 9$

Next

Measure how wrong $\hat{y}$ is against the true one-hot target y (step 1) with the cross-entropy loss — the single number that training will minimize.

6 Cross-entropy loss

The forward pass (step 5) ends with a predicted distribution $\hat{y}$; step 1 gave the true one-hot target y. We now need one scalar that says how wrong the prediction is — differentiable, so that training can push it down. That scalar is the cross-entropy.

6.1 One example

Cross-entropy between the target y and the prediction $\hat{y}$ is

$$\mathcal{L}_n = -\sum_{i=0}^{9} y_i \log \hat{y}_i,$$

where y_i is the one-hot target entry, $\hat{y}_i$ the predicted probability of class i, and log is the *natural* logarithm (base e — it pairs with the e in softmax). $\mathcal{L}_n$ is the loss on example n, a scalar.

Because y is one-hot — $y_t = 1$ for the true class t and $y_i = 0$ otherwise — the sum collapses. Every term with $i \neq t$ is $0 \cdot \log \hat{y}_i = 0$ (finite, since softmax guarantees $\hat{y}_i > 0$, so $\log \hat{y}_i$ never blows up), leaving only $i = t$:

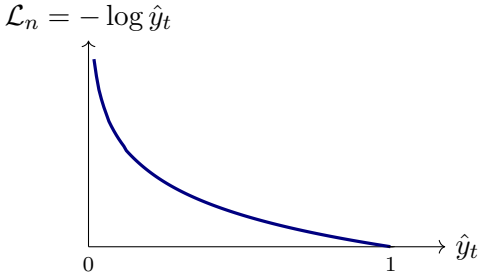
$$\boxed{\mathcal{L}_n = -\log \hat{y}_t}$$

The loss is just the negative log-probability the model assigned to the *correct* digit.

6.2 What it rewards

- Confident and correct ($\hat{y}_t \to 1$): $-\log 1 = 0$. No loss.
- Low probability on the truth ($\hat{y}_t \to 0$): $-\log(\to 0) \to +\infty$. Unbounded loss.

So it punishes confident wrong answers hard, and its range is $[0, \infty)$.



6.3 Why cross-entropy (not squared error)

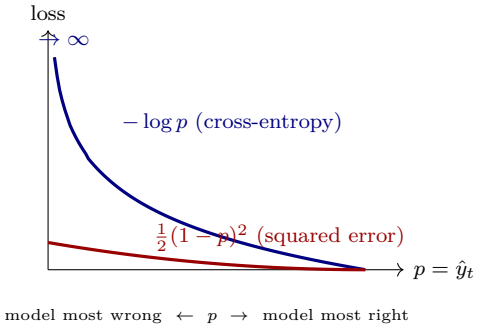
The natural alternative is plain squared error on the softmax output,

$$\mathcal{L}^{\text{SE}} = \tfrac{1}{2} \sum_{i=0}^{9} \big(\hat{y}_i - y_i\big)^2.$$

Both look sensible: each is 0 when the prediction matches the target and grows as it drifts away. What separates them is not their *value* but their *steepness* — how hard each one pushes to correct a mistake — because training moves the weights along the slope of the loss, never its value.

Compare them as functions of one number, $p := \hat{y}_t$, the probability the model gives the *true* class ($p = 1$ is perfectly right, $p \approx 0$ is confidently wrong). Keeping just the true-class term, the two losses are

$$\text{cross-entropy: } -\log p, \qquad \text{squared error: } \tfrac{1}{2}(1-p)^2.$$



The curves part company at the left edge, where the model is most wrong ($p \to 0$):

- Cross-entropy $-\log p$ shoots up without bound and keeps getting steeper. A steep loss is a strong correction, so the model is pushed hard to fix exactly the cases it gets badly wrong.
- Squared error $\frac{1}{2}(1-p)^2$ only climbs to $\frac{1}{2}$ and then flattens. A flat loss is a weak correction — it barely reacts to a confident mistake.

Softmax, sitting in front of the loss, makes this worse for squared error: it too goes insensitive at the extremes, so squared error's already-weak signal there shrinks further and learning can stall. Cross-entropy's blow-up is exactly steep enough to cancel that effect — its gradient with respect to the logits comes out to the clean $\hat{y} - y$ (derived just below, §6.5).

Cross-entropy also has a probabilistic reading: $-\log p$ is the negative log-likelihood of the true label, so minimizing it maximizes the likelihood the model gives the correct answers.

6.4 The batch

Start from the per-example loss in its full (un-collapsed) form from the one-image case above, now written with batch indices:

$$\mathcal{L}_n = -\sum_{i=0}^{9} Y_{ni} \log \hat{Y}_{ni},$$

where Y_{ni} is the one-hot target for example n, class i (row n of Y), and $\hat{Y}_{ni}$ is the predicted probability (row n of $\hat{Y}$).

Average over the mini-batch:

$$\mathcal{L} = \frac{1}{N}\sum_{n=0}^{N-1} \mathcal{L}_n = -\frac{1}{N}\sum_{n=0}^{N-1}\sum_{i=0}^{9} Y_{ni} \log \hat{Y}_{ni}.$$

Now collapse the *inner* sum exactly as in the one-image case above, once per row. For example n the target is one-hot: $Y_{ni} = 1$ only at $i = t_n$ (its true class) and 0 everywhere else, so every term but $i = t_n$ drops:

$$\sum_{i=0}^{9} Y_{ni} \log \hat{Y}_{ni} = Y_{n,t_n} \log \hat{Y}_{n,t_n} = \log \hat{Y}_{n,t_n} = \log \hat{y}^{(n)}_{t_n}.$$

Substituting this back into the average gives the compact form:

$$\mathcal{L} = -\frac{1}{N}\sum_{n=0}^{N-1} \log \hat{y}^{(n)}_{t_n},$$

where $t_n \in \{0, \ldots, 9\}$ is the true class of example n and $\hat{y}^{(n)}_{t_n} = \hat{Y}_{n,t_n}$ is the probability example n gave its own correct digit. The two forms are the *same* loss: the double sum is what vectorized code computes, the single sum is what makes the meaning plain.

We average rather than sum so the loss scale is independent of the batch size N — the learning rate then does not need retuning when N changes.

Notation flags. $\mathcal{L}$ (calligraphic) is the loss; do not confuse it with the plain L used informally in step 4 for "number of layers." And t_n is the label of *example* n — the per-example version of the single-image t from step 1.

6.5 The gradient of the loss

Training (step 7) shrinks $\mathcal{L}$ by gradient descent, so it needs the gradient. The cleanest place to begin is the gradient with respect to the *logits* $z^{(2)}$ (the scores fed into softmax, step 5), because softmax and cross-entropy collapse into something strikingly simple.

Work with a single example. Its ten logits form a vector

$$z := z^{(2)} = (z_0,\, z_1,\, \ldots,\, z_9) \in \mathbb{R}^{10},$$

where each component $z_m \in \mathbb{R}$ is the class-m logit (a real number, any sign — not a 0/1, not a probability). From these $\hat{y} = \mathrm{softmax}(z)$ is the prediction, y the one-hot target, and $\mathcal{L}_n = -\sum_i y_i \log \hat{y}_i$. We want $\partial \mathcal{L}_n / \partial z_m$ for each class $m \in \{0, \ldots, 9\}$.

(1) Chain rule through $\hat{y}$. z_m reaches the loss only through the probabilities, so

$$\frac{\partial \mathcal{L}_n}{\partial z_m} = \sum_{i=0}^{9} \frac{\partial \mathcal{L}_n}{\partial \hat{y}_i}\,\frac{\partial \hat{y}_i}{\partial z_m}.$$

(2) Loss with respect to one prediction. Differentiate $\mathcal{L}_n = -\sum_i y_i \log \hat{y}_i$ term by term, with $\frac{d}{du}\log u = \frac{1}{u}$. Since y is one-hot — $y_t = 1$ at the true class t (the index of the correct digit), and $y_i = 0$ for every $i \neq t$ —

$$\frac{\partial \mathcal{L}_n}{\partial \hat{y}_i} = -\frac{y_i}{\hat{y}_i} = \begin{cases} -\dfrac{1}{\hat{y}_i} & i = t, \\ 0 & i \neq t. \end{cases}$$

This is one scalar per class i; the ten together form the 10-vector $\partial \mathcal{L}_n / \partial \hat{y}$, nonzero only at t.

A worked number (full ten classes). Say the true digit is $t = 3$, so y is one-hot at index 3, and suppose the model predicts $\hat{y}$:

$$y = (0, 0, 0, 1, 0, 0, 0, 0, 0, 0),$$
$$\hat{y} = (0.05,\ 0.05,\ 0.10,\ 0.50,\ 0.05,\ 0.05,\ 0.05,\ 0.05,\ 0.05,\ 0.05).$$

Applying step (2) entry by entry, $-y_i/\hat{y}_i$ is 0 at every class with $y_i = 0$ (nine of them) and $-1/\hat{y}_3 = -1/0.50 = -2$ at the true class:

$$\frac{\partial \mathcal{L}_n}{\partial \hat{y}} = (0, 0, 0, -2, 0, 0, 0, 0, 0, 0).$$

A *ten*-vector (same shape as $\hat{y}$) with a single nonzero entry — $-1/\hat{y}_t = -2$, sitting at $t = 3$ — *not* the lone scalar -2. For contrast, the end result of the full derivation is

$$\hat{y} - y = (0.05,\ 0.05,\ 0.10,\ -0.50,\ 0.05,\ 0.05,\ 0.05,\ 0.05,\ 0.05,\ 0.05).$$

(3) Softmax with respect to one logit. First, the relationship between $\hat{y}_i$ and the logits. Softmax (step 5) is, written out,

$$\hat{y}_i = \frac{e^{z_i}}{S}, \qquad S = \sum_{l=0}^{9} e^{z_l}.$$

$\hat{y}_i$ depends on z_m in *two* places: through the numerator e^{z_i} (but only when $i = m$) and through the denominator S (always, since S adds up *every* logit). That double dependence is why the answer will have two terms.

We need two small derivatives first.

Numerator. The logits are independent inputs, so $\partial z_i/\partial z_m$ is 1 when $i = m$ and 0 otherwise — this is the Kronecker delta δ_{im}. By the chain rule,

$$\frac{\partial e^{z_i}}{\partial z_m} = e^{z_i}\,\frac{\partial z_i}{\partial z_m} = e^{z_i}\,\delta_{im}.$$

Denominator. Differentiate $S = \sum_l e^{z_l}$ term by term; only the $l = m$ term contains z_m, so all others vanish:

$$\frac{\partial S}{\partial z_m} = \sum_l e^{z_l}\,\delta_{lm} = e^{z_m}.$$

Now the quotient rule. For $\hat{y}_i = f/g$ with $f = e^{z_i}$ and $g = S$,

$$\frac{\partial}{\partial z_m}\frac{f}{g} = \frac{f'\,g - f\,g'}{g^2}, \qquad f' = \frac{\partial f}{\partial z_m},\quad g' = \frac{\partial g}{\partial z_m}.$$

Substitute $f' = e^{z_i}\delta_{im}$, $g = S$, $g' = e^{z_m}$:

$$\frac{\partial \hat{y}_i}{\partial z_m} = \frac{\left(e^{z_i}\delta_{im}\right) S - e^{z_i}\left(e^{z_m}\right)}{S^2}.$$

Split into two fractions and cancel one S in each:

$$= \frac{e^{z_i}\delta_{im}\,S}{S^2} - \frac{e^{z_i}e^{z_m}}{S^2} = \frac{e^{z_i}}{S}\,\delta_{im} - \frac{e^{z_i}}{S}\cdot\frac{e^{z_m}}{S}.$$

Finally read off the softmax values $\dfrac{e^{z_i}}{S} = \hat{y}_i$ and $\dfrac{e^{z_m}}{S} = \hat{y}_m$:

$$\frac{\partial \hat{y}_i}{\partial z_m} = \hat{y}_i\,\delta_{im} - \hat{y}_i\,\hat{y}_m = \hat{y}_i\big(\delta_{im} - \hat{y}_m\big).$$

Notation flag. δ_{im} is the *Kronecker delta* (1 if $i = m$, else 0) — a fixed number, not a gradient.

(4) Combine. Substitute (2) and (3); the $\hat{y}_i$ cancels — this cancellation is exactly the "cleanness" that 6.3 promised:

$$\frac{\partial \mathcal{L}_n}{\partial z_m} = \sum_i \Big(-\frac{y_i}{\hat{y}_i}\Big)\hat{y}_i\big(\delta_{im} - \hat{y}_m\big) = -\sum_i y_i\big(\delta_{im} - \hat{y}_m\big).$$

(5) Simplify. Split the sum, using $\sum_i y_i \delta_{im} = y_m$ (only $i = m$ survives) and $\sum_i y_i = 1$ (y is one-hot):

$$-\sum_i y_i \delta_{im} + \hat{y}_m \sum_i y_i = -y_m + \hat{y}_m \cdot 1 = \hat{y}_m - y_m.$$

Stacking over the ten classes,

$$\boxed{\frac{\partial \mathcal{L}_n}{\partial z^{(2)}} = \hat{y} - y \in \mathbb{R}^{10}.}$$

Shape check. The gradient of a scalar by the 10-vector $z^{(2)}$ is a 10-vector, and $\hat{y} - y$ is exactly that — prediction minus target, zero only when $\hat{y} = y$. (Squared error has no $1/\hat{y}_i$ to cancel the Jacobian's $\hat{y}_i$, which is why it gets no such tidy gradient.)

By-products. Step (2) is itself the gradient with respect to the prediction, $\partial \mathcal{L}_n / \partial \hat{y}_i = -y_i/\hat{y}_i$. And averaging over the mini-batch (the $\frac{1}{N}$ in the mean loss) gives the batched form

$$\frac{\partial \mathcal{L}}{\partial Z^{(2)}} = \frac{1}{N}\big(\hat{Y} - Y\big) \in \mathbb{R}^{N \times 10},$$

which is where backpropagation (step 7) begins.

6.6 Symbols

New persistent symbol this step — $\mathcal{L}$ — is added to the global list of symbols. The index letters below are *local to this section.*

Local symbol	Meaning (this section)
n	example index in the batch, $0, \ldots, N-1$
i, l	class indices, summed, $0, \ldots, 9$
m	the class whose logit we differentiate by, $0, \ldots, 9$
t_n	true class *of example* n, $\in \{0, \ldots, 9\}$
p	shorthand for $\hat{y}_t$, the probability on the true class
z	shorthand for the logits $z^{(2)}$; $z_m \in \mathbb{R}$ = class-m logit
S	softmax denominator $\sum_l e^{z_l}$
δ_{im}	Kronecker delta (1 if $i = m$, else 0)
$\mathcal{L}^{\mathrm{SE}}$	squared-error loss (the rejected alternative)

Next

Backpropagation (step 7): take the output gradient $\hat{y} - y$ just derived and propagate it back through the layers to reach the parameter gradients for $W^{(1)}$, $b^{(1)}$, $W^{(2)}$, $b^{(2)}$.

7 Backpropagation

Training needs the gradient of the loss with respect to every parameter: $\partial\mathcal{L}/\partial W^{(1)}$, $\partial\mathcal{L}/\partial b^{(1)}$, $\partial\mathcal{L}/\partial W^{(2)}$, $\partial\mathcal{L}/\partial b^{(2)}$. Backpropagation gets them by applying the chain rule from the loss backward through the network, reusing each layer's result in the one before it.

The starting point — the gradient of the loss at the output logits — was already derived in step 6: for one example $\partial\mathcal{L}_n/\partial z^{(2)} = \hat{y} - y$, and averaged over the mini-batch,

$$g^{(2)} := \frac{\partial\mathcal{L}}{\partial Z^{(2)}} = \frac{1}{N}\big(\hat{Y} - Y\big) \in \mathbb{R}^{N\times 10}.$$

$g^{(2)}_{ni}$ is how much the loss changes if the logit $Z^{(2)}_{ni}$ (example n, class i) is nudged — the $N\times 10$ matrix we propagate backward from. **Naming.** We write each layer's gradient-at-the-pre-activation as $g^{(\ell)}$, deliberately *not* $\delta^{(\ell)}$, to avoid colliding with the Kronecker δ_{im} in step 6's derivation.

7.1 Backward through the second linear layer

From $g^{(2)}$ we want three things — the gradients for $W^{(2)}$ and $b^{(2)}$ (to update them), and the gradient handed back to $A^{(1)}$ (to keep going into earlier layers). Recall the layer's forward rule, one entry (step 5):

$$Z^{(2)}_{ni} = \sum_{k=0}^{127} A^{(1)}_{nk}\, W^{(2)}_{ki} + b^{(2)}_i.$$

Gradient for $W^{(2)}$. The weight $W^{(2)}_{ki}$ appears only in the class-i logit, but in that logit for *every* example n. So by the chain rule we add up its effect over all examples (only over n — no other class contains it):

$$\frac{\partial\mathcal{L}}{\partial W^{(2)}_{ki}} = \sum_{n=0}^{N-1} \frac{\partial\mathcal{L}}{\partial Z^{(2)}_{ni}}\,\frac{\partial Z^{(2)}_{ni}}{\partial W^{(2)}_{ki}}.$$

From the forward rule the inner derivative is $\partial Z^{(2)}_{ni}/\partial W^{(2)}_{ki} = A^{(1)}_{nk}$ (the activation it multiplies), so

$$\frac{\partial\mathcal{L}}{\partial W^{(2)}_{ki}} = \sum_n A^{(1)}_{nk}\, g^{(2)}_{ni}.$$

That sum over n is exactly one entry of a matrix product. Stacking over all k and i,

$$\frac{\partial\mathcal{L}}{\partial W^{(2)}} = \big(A^{(1)}\big)^\top g^{(2)} \in \mathbb{R}^{128\times 10}.$$

Shape check. $(A^{(1)})^\top$ is $128 \times N$ and $g^{(2)}$ is $N \times 10$, giving 128×10 — the shape of $W^{(2)}$, as a gradient with respect to $W^{(2)}$ must be.

Gradient for $b^{(2)}$. The bias $b^{(2)}_i$ is added to logit i of every example, with $\partial Z^{(2)}_{ni}/\partial b^{(2)}_i = 1$, so

$$\frac{\partial\mathcal{L}}{\partial b^{(2)}_i} = \sum_n \frac{\partial\mathcal{L}}{\partial Z^{(2)}_{ni}} \cdot 1 = \sum_n g^{(2)}_{ni}.$$

In words: the bias gradient is $g^{(2)}$ summed down its rows (over the examples),

$$\frac{\partial\mathcal{L}}{\partial b^{(2)}} = \sum_{n=0}^{N-1} g^{(2)}_{n,:} \in \mathbb{R}^{10}.$$

Gradient handed back to $A^{(1)}$. To reach the earlier layers we need how the loss depends on the hidden activations. An activation $A^{(1)}_{nk}$ (example n, hidden unit k) feeds into all ten logits of *that same example*, so this time we sum over the classes i:

$$\frac{\partial \mathcal{L}}{\partial A^{(1)}_{nk}} = \sum_{i=0}^{9} \frac{\partial \mathcal{L}}{\partial Z^{(2)}_{ni}} \frac{\partial Z^{(2)}_{ni}}{\partial A^{(1)}_{nk}} = \sum_{i} g^{(2)}_{ni} W^{(2)}_{ki},$$

using $\partial Z^{(2)}_{ni} / \partial A^{(1)}_{nk} = W^{(2)}_{ki}$. In matrix form,

$$\frac{\partial \mathcal{L}}{\partial A^{(1)}} = g^{(2)} \big(W^{(2)}\big)^{\top} \in \mathbb{R}^{N \times 128}.$$

Shape check. $g^{(2)}$ is $N \times 10$ and $(W^{(2)})^{\top}$ is 10×128, giving $N \times 128$ — the shape of $A^{(1)}$.

7.2 Backward through ReLU

Now turn $\partial \mathcal{L}/\partial A^{(1)}$ into $g^{(1)} := \partial \mathcal{L}/\partial Z^{(1)}$, the gradient at the hidden *pre*-activation. ReLU acts entry by entry, $A^{(1)}_{nk} = \max(0, Z^{(1)}_{nk})$, with derivative (step 3; the kink at 0 folded into the lower case)

$$\frac{\partial A^{(1)}_{nk}}{\partial Z^{(1)}_{nk}} = \begin{cases} 1 & Z^{(1)}_{nk} > 0 \\ 0 & Z^{(1)}_{nk} \le 0 \end{cases} =: M_{nk},$$

and there are no cross-terms (entry nk depends only on entry nk). The chain rule is then a plain entry-by-entry multiply:

$$g^{(1)}_{nk} = \frac{\partial \mathcal{L}}{\partial A^{(1)}_{nk}} M_{nk}.$$

The matrix M (each entry 0 or 1, shape $N \times 128$) is the *mask* foreshadowed in step 3: it lets the gradient through where the unit was active ($Z^{(1)}_{nk} > 0$) and blocks it where the unit was off. Writing $\odot$ for the entry-by-entry (Hadamard) product and substituting $\partial \mathcal{L}/\partial A^{(1)}$ from above,

$$g^{(1)} = \Big(g^{(2)} \big(W^{(2)}\big)^{\top} \Big) \odot M \in \mathbb{R}^{N \times 128}.$$

7.3 Backward through the first linear layer

Layer 1, $Z^{(1)} = XW^{(1)} + b^{(1)}$ (step 2), has exactly the same form as layer 2 — only the names change: the input is X instead of $A^{(1)}$, the weights are $W^{(1)}$, and the incoming gradient is $g^{(1)}$ instead of $g^{(2)}$. So the three derivations above repeat verbatim and give, with no new work,

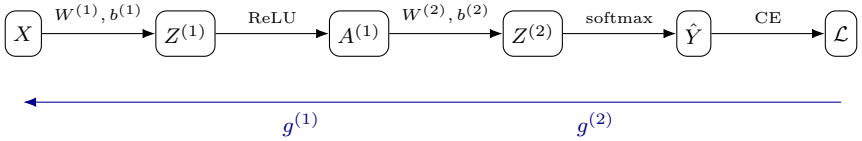
$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = X^{\top} g^{(1)} \in \mathbb{R}^{784 \times 128}, \qquad \frac{\partial \mathcal{L}}{\partial b^{(1)}} = \sum_{n=0}^{N-1} g^{(1)}_{n,:} \in \mathbb{R}^{128}.$$

We could also form $\partial \mathcal{L}/\partial X = g^{(1)} (W^{(1)})^{\top}$, but X is the fixed input data — nothing before it is trained — so we stop here.

7.4 The whole backward pass

Collect the six results in the order they are computed. The forward pass already stored X, $Z^{(1)}$, $A^{(1)}$, $Z^{(2)}$, $\hat{Y}$; the backward pass walks right to left and reuses them:

#	Quantity	Shape
1	$g^{(2)} = \frac{1}{N}(\hat{Y} - Y)$	$N \times 10$
2	$\partial\mathcal{L}/\partial W^{(2)} = (A^{(1)})^\top g^{(2)}$	128×10
3	$\partial\mathcal{L}/\partial b^{(2)} = \sum_n g^{(2)}_{n,:}$	10
4	$g^{(1)} = \big(g^{(2)}(W^{(2)})^\top\big) \odot M$	$N \times 128$
5	$\partial\mathcal{L}/\partial W^{(1)} = X^\top g^{(1)}$	784×128
6	$\partial\mathcal{L}/\partial b^{(1)} = \sum_n g^{(1)}_{n,:}$	128



Forward pass (top, left → right) builds the prediction; the backward pass (bottom, right → left) carries the gradient — it is $g^{(2)}$ at the logits and $g^{(1)}$ at the hidden pre-activation, and at each pre-activation it also spawns that layer's weight and bias gradients.

The pattern — and the reason it is called *back*propagation — is that each layer's gradient $g^{(\ell)}$ is computed once from the layer after it, then reused both for that layer's weight and bias gradients *and* to make the next g one step further back. Nothing is recomputed from scratch; the cost of all the gradients is about that of one extra forward pass.

7.5 Symbols

New persistent symbols this step — $g^{(2)}$ and $g^{(1)}$, the gradients of the loss with respect to each layer's pre-activation — are added to the global list of symbols. The symbols below are *local to this section.*

Local symbol	Meaning (this section)
n	example index in the batch, $0, \ldots, N-1$
i	class index, $0, \ldots, 9$
k	hidden-unit index, $0, \ldots, 127$
M	ReLU 0/1 mask, $M_{nk} = [Z^{(1)}_{nk} > 0]$; $N \times 128$
$\odot$	entry-by-entry (Hadamard) product

Next

Use these six gradients to improve the parameters: one step of mini-batch SGD,

$$W \leftarrow W - \eta \frac{\partial \mathcal{L}}{\partial W} \qquad \text{(and likewise for each bias)},$$

with learning rate η.

8 Updating the parameters: mini-batch SGD

Backprop (step 7) handed us the six gradients — how the loss changes with each parameter. Now we use them to make the loss smaller. The method is *gradient descent*: repeatedly nudge each parameter a little way *downhill* (opposite its gradient). Because each step's gradient comes from a random mini-batch (step 6) rather than the whole dataset, the full name is *stochastic gradient descent* (SGD) — "stochastic" meaning "driven by random samples" — and "mini-batch SGD" just says the batch holds N examples. We build the rule up in that order: one step first, then the mini-batch part.

8.1 One gradient-descent step

The gradient $\partial\mathcal{L}/\partial\theta$ of a parameter θ points in the direction that *increases* the loss fastest. To *decrease* the loss, step the opposite way:

$$\theta \;\leftarrow\; \theta - \eta\,\frac{\partial\mathcal{L}}{\partial\theta}.$$

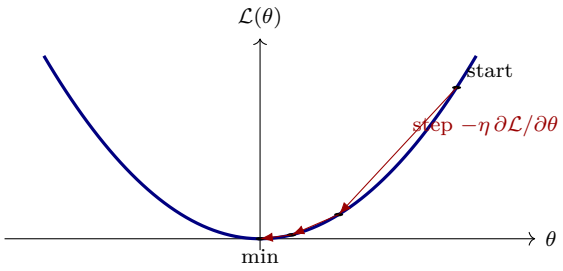
Here θ stands for any one parameter, $\leftarrow$ means "replace with," and $\eta > 0$ is the *learning rate* — how big a step to take (a hyperparameter; our running value is $\eta = 0.1$).

Apply the same rule to all four parameter blocks, using the gradients from step 7:

$$W^{(2)} \leftarrow W^{(2)} - \eta\,\frac{\partial\mathcal{L}}{\partial W^{(2)}}, \qquad b^{(2)} \leftarrow b^{(2)} - \eta\,\frac{\partial\mathcal{L}}{\partial b^{(2)}},$$
$$W^{(1)} \leftarrow W^{(1)} - \eta\,\frac{\partial\mathcal{L}}{\partial W^{(1)}}, \qquad b^{(1)} \leftarrow b^{(1)} - \eta\,\frac{\partial\mathcal{L}}{\partial b^{(1)}}.$$

Each gradient has the same shape as its parameter (step 7), so every subtraction is entry by entry — well defined, nothing to reshape.

8.2 Why a small step, repeated



The gradient is only the *local* downhill direction, valid only at the current point; take too big a step and you can shoot past the bottom. So we take a small step, recompute the gradient where we land, and repeat. The learning rate sets the size:

- Too small: many tiny steps, slow training.
- Too large: steps overshoot; the loss oscillates or even grows.
- Just right: a steady decrease toward a minimum.

8.3 Why "mini-batch" and "stochastic"

The loss we differentiated is the average over a mini-batch of N examples (step 6), not the whole training set. So our gradient is an *estimate* of the true gradient (the one over all 60,000 images), computed from a random handful — that is the "stochastic" in stochastic gradient descent (SGD). The size N trades off:

- $N = 1$ (pure SGD): cheap per step, but a very noisy gradient.
- $N =$ all 60,000 (full batch): the exact gradient, but slow and memory-hungry per step.
- N a few dozen to a few hundred (mini-batch): a good gradient estimate at low cost — the standard choice, and the leftover noise even helps escape poor spots.

One full pass through the training set — $60{,}000/N$ mini-batch steps — is called an *epoch*.

8.4 Symbols

New persistent symbol this step — η, the learning rate — is added to the global list of symbols. The symbols below are *local to this section*.

Local symbol	Meaning (this section)
θ	any single parameter (stands for $W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}$)
$\leftarrow$	"replace the left side with the right side" (an update)
epoch	one full pass through the training set, $60{,}000/N$ steps

Next

Put it together into the training loop: repeatedly draw a mini-batch and run forward → loss → backward → update, for many epochs, until the loss stops improving.

9 The training loop

Every piece is now in hand: the forward pass (step 2–step 5), the loss (step 6), the gradients (step 7), and the update rule (step 8). Training is just running them in a loop.

9.1 Initialization

Before the loop we set the starting parameters. Biases start at zero; weights start at small random values drawn with *He initialization* (the scheme promised back in step 2 and step 3, suited to ReLU):

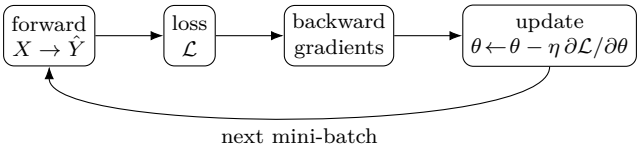
$$W_{ki}^{(\ell)} \sim \mathcal{N}\Big(0,\ \tfrac{2}{\text{fan-in}_\ell}\Big), \qquad b^{(\ell)} = 0.$$

Here $\mathcal{N}(\mu, \sigma^2)$ is a Gaussian (mean μ, variance σ^2), drawn independently for each weight, and fan-in_ℓ is the number of inputs to layer ℓ: $\text{fan-in}_1 = 784$ and $\text{fan-in}_2 = 128$. Scaling the variance by 2/fan-in keeps the pre-activations from growing or shrinking as they pass through the layer.

Why random and not all zero? If every weight were identical, all hidden units would compute the same thing and receive the same gradient, so they would stay identical forever — the random start breaks that symmetry and lets units specialize.

9.2 The loop

1. Initialize $W^{(1)}, W^{(2)}$ (He) and $b^{(1)}, b^{(2)} = 0$.
2. For each epoch $1, \ldots, E$:
   (a) Shuffle the training examples.
   (b) For each mini-batch of N examples (X, Y):
      i. **Forward:** $X \to Z^{(1)} \to A^{(1)} \to Z^{(2)} \to \hat{Y}$ (step 2–step 5).
      ii. **Loss:** $\mathcal{L} = \text{cross-entropy}(\hat{Y}, Y)$ (step 6) — recorded for monitoring.
      iii. **Backward:** the six gradients (step 7).
      iv. **Update:** $\theta \leftarrow \theta - \eta\, \partial\mathcal{L}/\partial\theta$ for all parameters (step 8).



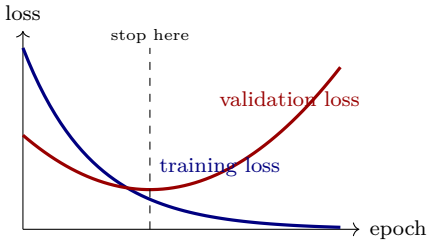
9.3 The full running example

Collecting every choice we have made along the way:

Setting	Value
Architecture	$784 \to 128 \to 10$
Hidden activation	ReLU
Output	softmax
Loss	cross-entropy
Optimizer	mini-batch SGD
Learning rate η	0.1
Batch size N	64
Epochs E	$\sim$20
Initialization	He (weights), 0 (biases)

9.4 Monitoring and when to stop

Watch the loss as training runs. The *training* loss should fall steadily. But the goal is performance on data the network has *not* seen, so we also check a held-out *validation* split each epoch. The two can diverge: past some point the training loss keeps dropping while the validation loss starts rising — the network is memorizing the training set (overfitting) instead of generalizing. Stop near the validation minimum (*early stopping*).



We reserved the 10,000-image test set (step 1) for one final, honest accuracy number; the validation split (carved from the training data) is what we peek at to decide when to stop.

9.5 What you get

Each update nudges all ∼101,770 parameters (100,480 in layer 1 + 1,290 in layer 2) a little; over thousands of mini-batch steps the weights settle into values that separate the ten digits. This plain two-layer network reaches roughly 97–98% test accuracy on MNIST. Larger gains (convolutional networks, better optimizers, regularization) build on exactly this skeleton — forward, loss, backward, update, repeat.

9.6 Symbols

New persistent symbol this step — E, the number of epochs — is added to the global list of symbols. The symbols below are *local to this section.*

Local symbol	Meaning (this section)
$\mathcal{N}(\mu, \sigma^2)$	Gaussian with mean μ, variance σ^2
fan-in$_\ell$	number of inputs to layer ℓ (784, then 128)
ℓ	layer index, 1 or 2
k, i	weight row / column index within a layer